PROGRAMACIÓN DIDÁCTICA: ACCESO A DATOS (0846)

Resumen de Unidades de Programación (Curso 2026-2027)

Código	Título de la UP
UP01	**ACCESO A FICHEROS**
UP02	**ACCESO A BBDD RELACIONALES**
UP03	**MAPEO DE OBJETOS RELACIONAL (ORM)**
UP04	**CONSUMIR SERVICIOS WEB (API'S)**
UP05	**ACCESO A BD NoSQL**

- La suma total de estas unidades es de **133 horas**, correspondientes íntegramente al periodo de formación realizado en el centro educativo fuera del periodo dual.

UP01: ACCESO A FICHEROS

1. Identificación

Campo	Detalle
Código	UP01
Módulo	Acceso a datos (0846)
Duración	**10 Horas**
Temporalización	Del **11/09/2026** al **25/09/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA01.** Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas.

Criterios de Evaluación * **a)** Se han utilizado clases para la gestión de ficheros y directorios. * **b)** Se han valorado las ventajas y los inconvenientes de las distintas formas de acceso. * **c)** Se han utilizado clases para recuperar información almacenada en ficheros. * **d)** Se han utilizado clases para almacenar información en ficheros. * **e)** Se han utilizado clases para realizar conversiones entre

diferentes formatos de ficheros. * **f)** Se han previsto y gestionado las excepciones. * **g)** Se han probado y documentado las aplicaciones desarrolladas.

Competencias * **Profesionales:** b), e), f), l), r). * **Ocupación:** b), e), f), l), r).

3. Organización

Contenidos * Tipos de ficheros: ficheros de texto y ficheros binarios. * Ficheros de texto: texto plano, ficheros de configuración y ficheros XML. * Acceso a ficheros: abrir y cerrar un fichero, modos de apertura (`r`, `w`, `a`, `+`) y procesamiento línea a línea. * Escritura en ficheros y agregado de información al final (ficheros de registro o *log*). * Manipulación de ficheros en forma binaria: lectura/escritura de bytes y posicionamiento (`DataInput`, `seek`). * Persistencia de datos y serialización: ficheros CSV (módulo `csv`) y ficheros binarios (módulo `pickle`). * Gestión de directorios independiente del sistema (módulo `os`, `os.path.join`). * La sentencia `with` para la apertura y cierre automático de recursos. * Procesamiento de ficheros XML con `ElementTree`: lectura, escritura, modificación y borrado de elementos y atributos. * Previsión y gestión de excepciones en las operaciones de E/S. * Pruebas y documentación de las aplicaciones desarrolladas.

Metodología * Se emplean metodologías activas centradas en el alumnado. El aprendizaje se organiza en torno a **prácticas guiadas de programación** en las que el estudiante construye, de forma incremental, una pequeña aplicación que lee y escribe información en ficheros de distintos formatos (texto, CSV, binario y XML).

Secuencia de actividades * **A1:** Identificación de tipos de ficheros y lectura/escritura de ficheros de texto plano. * **A2:** Modos de apertura, agregado de información (logs) y manipulación de ficheros binarios. * **A3:** Persistencia mediante CSV y `OpenCSV` y gestión de directorios. * **A4:** Procesamiento de ficheros XML (lectura, escritura, modificación y borrado), control de excepciones y documentación.

Recursos * Equipos del aula de informática, IDE de Java (VS Code / VS Codium), JVM, sus módulos estándar (`java.xml`, `java.io`), la librería externa (`OpenCSV`), control de versiones (Git) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA01. * **Rúbricas de prácticas (20%):** valoración sistemática de las aplicaciones desarrolladas en cada actividad, atendiendo a la corrección técnica, la gestión de excepciones y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP02: ACCESO A BBDD RELACIONALES

1. Identificación

Campo	Detalle
Código	UP02
Módulo	Acceso a datos (0846)
Duración	**10 Horas**
Temporalización	Del **28/09/2026** al **19/10/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA02.** Desarrolla aplicaciones que gestionan información almacenada en bases de datos relacionales identificando y utilizando mecanismos de conexión.

Criterios de Evaluación * **a)** Se han valorado las ventajas e inconvenientes de utilizar conectores. * **b)** Se han utilizado gestores de bases de datos embebidos e independientes. * **c)** Se ha utilizado el conector idóneo en la aplicación. * **d)** Se ha establecido la conexión. * **e)** Se ha definido la estructura de la base de datos. * **f)** Se han desarrollado aplicaciones que modifican el contenido de la base de datos. * **g)** Se han definido los objetos destinados a almacenar el resultado de las consultas. * **h)** Se han desarrollado aplicaciones que efectúan consultas. * **i)** Se han eliminado los objetos una vez finalizada su función. * **j)** Se han gestionado las transacciones. * **k)** Se han ejecutado procedimientos almacenados en la base de datos.

Competencias * **Profesionales:** c), e), p), r). * **Ocupación:** c), e), p), r).

3. Organización

Contenidos

- Repaso de bases de datos relacionales y SQL: tablas, columnas, filas, clave primaria y clave foránea, y tipos de relaciones (1:N, N:M).

- El lenguaje SQL: comandos DDL, DML, DCL y TCL.
- Sistemas Gestores de Bases de Datos (SGBD) embebidos e independientes: SQLite frente a MySQL/MariaDB.
- Conectividad con bases de datos desde Java mediante **JDBC (Java Database Connectivity)**: ventajas e inconvenientes; uso de la API `java.sql` y de los controladores JDBC para SQLite y MySQL/MariaDB; gestión de dependencias mediante Maven o incorporación de archivos JAR.
- Establecimiento y cierre de la conexión con la base de datos mediante los objetos `Connection` y los controladores JDBC.
- Definición de la estructura de la base de datos desde la aplicación mediante objetos `Statement` y `PreparedStatement`.
- Los objetos **Statement** y **PreparedStatement**: ejecución de consultas y actualización de datos. Uso de `ResultSet` para la recuperación de resultados.
- Operaciones CRUD: aplicaciones que modifican el contenido y aplicaciones que efectúan consultas.
- Objetos destinados a almacenar y recorrer el resultado de las consultas mediante `ResultSet`.
- Gestión de transacciones: propiedades ACID, `commit()` y `rollback()` mediante el objeto `Connection`.
- Ejecución de procedimientos almacenados mediante `CallableStatement`.
- Liberación de objetos y recursos una vez finalizada su función mediante `try-with-resources` y los métodos `close()`.

Metodología * Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado desarrolla aplicaciones Java que se conectan a un SGBD mediante JDBC, definen su estructura y realizan operaciones CRUD y transaccionales sobre los datos.

Secuencia de actividades * **A1:** Repaso de SQL y del modelo relacional; uso de gestores embebidos e independientes. * **A2:** Configuración de JDBC y de los controladores, establecimiento de la conexión y definición de la estructura de la BD. * **A3:** Operaciones CRUD mediante `Statement` y `PreparedStatement`, y almacenamiento y recorrido de los resultados de las consultas mediante `ResultSet`. * **A4:** Gestión de transacciones (ACID), ejecución de procedimientos almacenados mediante `CallableStatement` y liberación de recursos mediante `try-with-resources`.

Recursos * Equipos del aula de informática, IDE de Java, JDK, SQLite, servidor MySQL/MariaDB (XAMPP o Docker), API JDBC (`java.sql`), controladores JDBC para SQLite y MySQL/MariaDB, Maven para la gestión de dependencias, control de versiones (Git) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA02. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de acceso a la base de datos desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP03: MAPEO DE OBJETOS RELACIONAL (ORM)

1. Identificación

Campo	Detalle
Código	UP03
Módulo	Acceso a datos (0846)
Duración	**12 Horas**
Temporalización	Del **23/10/2026** al **09/11/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA03.** Gestiona la persistencia de los datos identificando herramientas de mapeo objeto relacional (ORM) y desarrollando aplicaciones que las utilizan.

Criterios de Evaluación * **a)** Se ha instalado la herramienta ORM. * **b)** Se ha configurado la herramienta ORM. * **c)** Se han definido configuraciones de mapeo. * **d)** Se han aplicado mecanismos de persistencia a los objetos. * **e)** Se han desarrollado aplicaciones que modifican y recuperan objetos persistentes. * **f)** Se han desarrollado aplicaciones que realizan consultas usando el lenguaje SQL. * **g)** Se han gestionado las transacciones.

Competencias * **Profesionales:** c), e), r). * **Ocupación:** c), e), r).

3. Organización

Contenidos * Concepto de ORM (Mapeo Objeto-Relacional): trabajar con bases de datos relacionales desde un enfoque de Programación Orientada a Objetos. * El desafío de los modelos relacional y orientado a objetos: la impedancia objeto-relacional. * Alternativas tradicionales (JDBC y consultas SQL) frente a ORM: ventajas e inconvenientes. * Instalación de la herramienta ORM mediante Maven o incorporación de las librerías necesarias. * Configuración de la herramienta ORM en una aplicación Java. * Definición de configuraciones de mapeo: correspondencia entre clases Java y tablas de la base de datos mediante anotaciones JPA. * Mecanismos de persistencia de objetos mediante JPA (Jakarta Persistence). * Desarrollo de aplicaciones que modifican y recuperan objetos persistentes mediante un ORM. * Consultas mediante JPQL (Jakarta Persistence Query Language), consultas SQL y Criteria API. * Gestión de transacciones mediante JPA y JDBC.

Metodología * Metodologías activas basadas en **prácticas de programación**. El alumnado refactoriza el acceso a datos visto en la UP02 sustituyendo el uso directo de JDBC por una herramienta ORM basada en JPA, comparando ambos enfoques.

Secuencia de actividades * **A1:** Introducción al ORM y comparación entre JDBC y el mapeo objeto-relacional. * **A2:** Instalación y configuración de la herramienta ORM mediante Maven y definición de las configuraciones de mapeo mediante anotaciones JPA. * **A3:** Persistencia, modificación y recuperación de objetos persistentes mediante JPA. * **A4:** Consultas con JPQL, Criteria API y SQL, y gestión de transacciones.

Recursos * Equipos del aula de informática, IDE de Java, JDK, Maven, JPA (Jakarta Persistence), Hibernate ORM como implementación de JPA, servidor MySQL/MariaDB, cliente de base de datos, control de versiones (Git) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA03. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones desarrolladas con la herramienta ORM en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP04: CONSUMIR SERVICIOS WEB (API'S)

1. Identificación

Campo	Detalle
Código	UP04
Módulo	Acceso a datos (0846)
Duración	**8 Horas**
Temporalización	Del **12/11/2026** al **23/11/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA04.** Desarrolla aplicaciones que gestionan la información almacenada en bases de datos objeto relacionales y orientadas a objetos valorando sus características y utilizando los mecanismos de acceso incorporados.

Criterios de Evaluación * **a)** Se han identificado las ventajas e inconvenientes de las bases de datos que almacenan objetos. * **b)** Se han establecido y cerrado conexiones. * **c)** Se ha gestionado la persistencia de objetos simples. * **d)** Se ha gestionado la persistencia de objetos estructurados. * **e)** Se han desarrollado aplicaciones que realizan consultas. * **f)** Se han modificado los objetos almacenados. * **g)** Se han gestionado las transacciones. * **h)** Se han probado y documentado las aplicaciones desarrolladas.

Competencias * **Profesionales:** c), e), r). * **Ocupación:** c), e), r).

3. Organización

Contenidos * Concepto de API (Interfaz de Programación de Aplicaciones): componentes clave y tipos de API (de bibliotecas, del sistema operativo y web). * REST (Representational State Transfer): principios y restricciones arquitectónicas (sin estado, interfaz uniforme, separación cliente-servidor, cacheabilidad). * La API REST como mecanismo de acceso a los datos: correspondencia entre los recursos de la API y los objetos (tablas y registros) de la base de datos relacional. * Métodos HTTP (GET, POST, PUT, DELETE) y el concepto de idempotencia, asociados a las operaciones sobre los objetos (consultar, crear, modificar y eliminar). * Formatos de intercambio de datos: JSON y XML como representación de objetos simples y estructurados. * Establecimiento y cierre de conexiones con el servicio mediante las clases de la API `java.net.http`. * Persistencia y recuperación de objetos

simples y estructurados a través de la API; modificación de los objetos almacenados. * Encabezados y parámetros de la solicitud: `Authorization`, `Content-Type`, `Accept`; parámetros de ruta y de consulta (*query string*). * Códigos de estado HTTP (200 OK, 404 Not Found, 500 Internal Server Error, …). * Gestión de transacciones, pruebas y documentación de las aplicaciones desarrolladas.

Metodología * Metodologías activas basadas en **prácticas de programación**. Partiendo de una base de datos relacional, el alumnado desarrolla y consume una API REST utilizando Java, enviando peticiones HTTP y procesando las respuestas en formato JSON para consultar y modificar los objetos expuestos por la API.

Secuencia de actividades * **A1:** Concepto de API y tipos de API; introducción a REST y sus restricciones; la API como mecanismo de acceso a los datos. * **A2:** Correspondencia recursos - objetos de la BD relacional; métodos HTTP, formatos JSON/XML, encabezados, parámetros y códigos de estado. * **A3:** Consumo de la API mediante `HttpClient`, `HttpRequest` y `HttpResponse`: establecimiento de conexiones, envío de peticiones y consulta y recuperación de objetos simples y estructurados. * **A4:** Modificación de los objetos almacenados mediante peticiones HTTP, gestión de transacciones, pruebas y documentación.

Recursos * Equipos del aula de informática, IDE de Java, JDK, API HTTP de Java (`java.net.http`), librerías para trabajar con JSON como Jackson o Gson, librerías para trabajar con XML como las incluidas en el JDK, cliente de pruebas de API (Postman), servicio REST de pruebas, Maven para la gestión de dependencias, control de versiones (Git) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA04. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de consumo de servicios web desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP05: ACCESO A BD NoSQL

1. Identificación

Campo	Detalle
Código	UP05
Módulo	Acceso a datos (0846)
Duración	**22 Horas**
Temporalización	Del **27/11/2026** al **29/01/2027**

2. Fundamentación

Resultados de Aprendizaje * **RA05.** Desarrolla aplicaciones que gestionan la información almacenada en bases de datos documentales nativas evaluando y utilizando clases específicas. * **RA06.** Programa componentes de acceso a datos identificando las características que debe poseer un componente y utilizando herramientas de desarrollo.

Criterios de Evaluación

RA05: * **a)** Se han valorado las ventajas e inconvenientes de utilizar bases de datos documentales nativas. * **b)** Se ha establecido la conexión con la base de datos. * **c)** Se han desarrollado aplicaciones que efectúan consultas sobre el contenido de la base de datos. * **d)** Se han añadido y eliminado colecciones de la base de datos. * **e)** Se han desarrollado aplicaciones para añadir, modificar y eliminar documentos de la base de datos.

RA06: * **a)** Se han valorado las ventajas e inconvenientes de utilizar programación orientada a componentes. * **b)** Se han identificado herramientas de desarrollo de componentes. * **c)** Se han programado componentes que gestionan información almacenada en ficheros. * **d)** Se han programado componentes que gestionan mediante conectores información almacenada en bases de datos. * **e)** Se han programado componentes que gestionan información usando mapeo objeto relacional. * **f)** Se han programado componentes que gestionan información almacenada en bases de datos objeto relacionales y orientadas a objetos. * **g)** Se han programado componentes que gestionan información almacenada en una base de datos documental nativa. * **h)** Se han probado y documentado los componentes desarrollados. * **i)** Se han integrado los componentes desarrollados en aplicaciones.

Competencias * **Profesionales:** c), e), l), p), r), t). * **Ocupación:** c), e), l), p), r), t).

3. Organización

Contenidos

Bases de datos documentales nativas (RA05): * Bases de datos NoSQL documentales: ventajas e inconvenientes frente a las bases de datos relacionales. * MongoDB: documentos JSON y esquema flexible. * Puesta en marcha de MongoDB (contenedor Docker), conexión al servidor y gestión de usuarios. * Bases de datos y colecciones: `use`, `createCollection`, `show dbs`, `show collections`. * Operaciones CRUD sobre documentos: `insertMany`, `find`, modificación, eliminación y `dropDatabase`. * Relaciones en MongoDB: documentos embebidos frente a referencias. * Conexión y desarrollo de aplicaciones Java que consultan y manipulan documentos mediante el controlador oficial de MongoDB para Java.

Programación de componentes de acceso a datos (RA06): * Programación orientada a componentes: ventajas e inconvenientes y herramientas de desarrollo. * Componentes de acceso a datos para ficheros, JDBC para bases de datos relacionales, ORM mediante JPA/Hibernate, bases de datos objeto-relacionales y bases de datos documentales nativas mediante MongoDB. * Pruebas, documentación e integración de los componentes desarrollados en aplicaciones Java.

Metodología * Metodologías activas basadas en **prácticas de programación** y en un proyecto integrador. El alumnado desarrolla aplicaciones Java sobre una base de datos documental (MongoDB) y, como cierre del módulo, programa e integra componentes reutilizables de acceso a datos que recogen todas las tecnologías trabajadas durante el curso.

Secuencia de actividades * **A1:** Bases de datos documentales nativas: características, puesta en marcha de MongoDB y conexión. * **A2:** Gestión de bases de datos y colecciones; operaciones CRUD sobre documentos. * **A3:** Desarrollo de aplicaciones Java que consultan y manipulan documentos mediante el controlador de MongoDB; relaciones (embebidos vs. referencias). * **A4:** Programación de componentes de acceso a datos (ficheros, JDBC, ORM, objeto-relacional y documental). * **A5:** Pruebas, documentación e integración de los componentes en una aplicación final.

Recursos

- Equipos del aula de informática, IDE de Java, JDK, MongoDB (contenedor Docker), cliente de base de datos (MongoDB Compass), controlador oficial de MongoDB para Java, JPA/Hibernate, JDBC, Maven para la gestión de dependencias, Docker, control de versiones (Git) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA05 y del RA06. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones sobre MongoDB y de los componentes de acceso a datos desarrollados. * **Pruebas objetivas (80%):** pruebas teórico-prácticas al finalizar cada RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas.